Fundamentos de Agentes de Inteligencia Artificial

Introducción

En los últimos años, la inteligencia artificial experimentó un crecimiento significativo, especialmente con la aparición de los modelos grandes de lenguajes (LLMs). Estas tecnologías permiten desarrollar sistemas capaces de comprender instrucciones en lenguaje natural, generar contenido, resumir documentos, escribir código y asistir en la resolución de problemas de distinta complejidad.

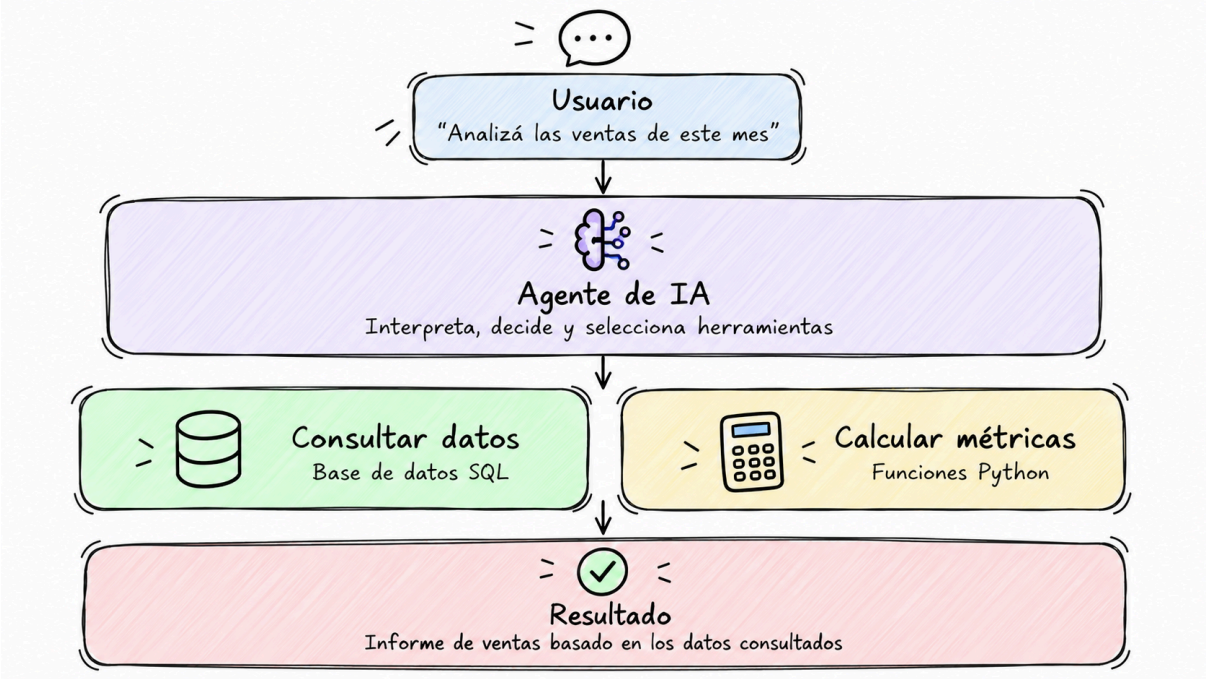
Sin embargo, a medida que comenzaron a incorporarse en aplicaciones reales, surgió una limitación importante: **generar una respuesta no siempre es suficiente para resolver un problema**, y es que, en muchos escenarios, un sistema necesita consultar información actualizada, interactuar con otras aplicaciones, ejecutar funciones, analizar resultados obtenidos y ejecutar múltiples operaciones para alcanzar un objetivo.

Y esto es justamente lo que impulsa al desarrollo de **Agentes de Inteligencia Artificial.**

1. ¿Que es un Agente de IA?

Los agentes de IA son sistemas de software que usan la IA para alcanzar objetivos y completar tareas en nombre de los usuarios. Muestran un razonamiento, planificación y memoria, y tienen un nivel de autonomía para tomar decisiones, aprender y adaptarse. Sus capacidades son posibles gracias a en gran parte a la capacidad multimodal de la IA generativa y los modelos básicos de IA.

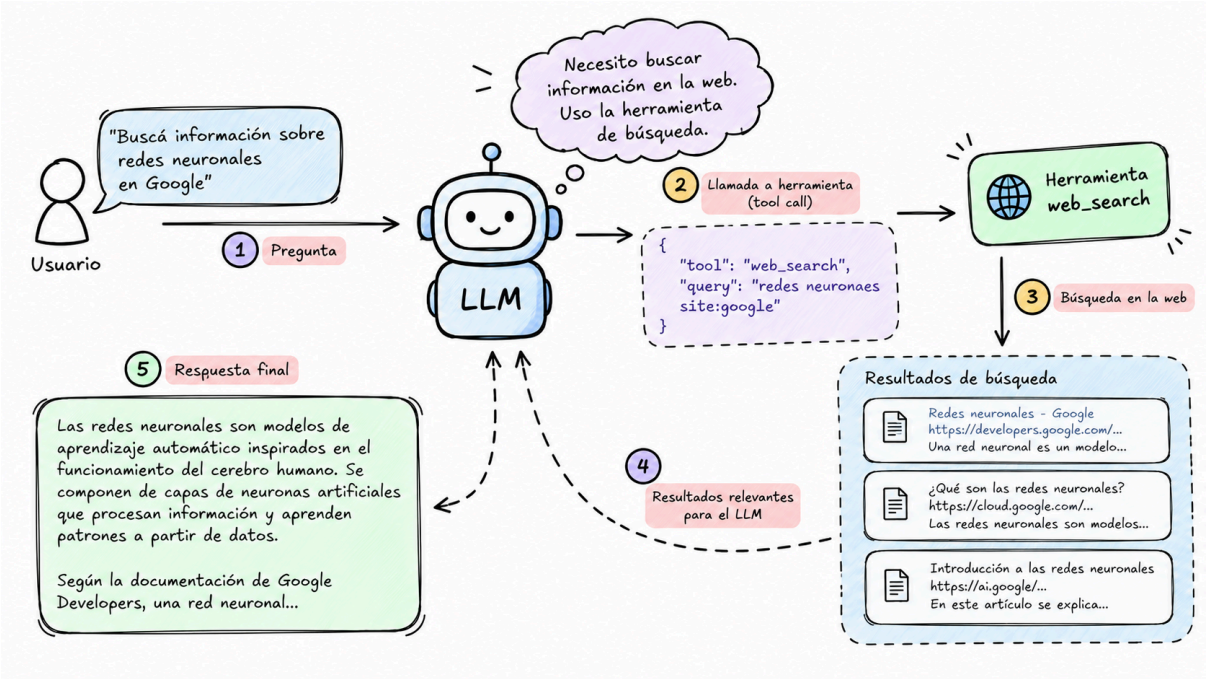
La idea en si de un agente de IA es que puede decidir que acción realizar dentro de un entorno definido por el desarrollador:



Algo a tener en cuenta es que, el modelo no ejecuta las consultas por sí mismo, sino que puede decidir qué **herramienta** necesita, solicitar su ejecución y utilizar el resultado para continuar. La misma aplicación controla qué herramientas existen y qué permisos tienen.

2. ¿Qué es una herramienta?

Las tools o herramientas son funciones, programas o conexiones externas que un modelo de IA puede utilizar para realizar acciones o consultar datos en tiempo real, más allá de solo generar texto. Mientras que un LLM (el cerebro) solo procesa información que fue entrenado, las tools le permiten "actuar" en el mundo digital. Por ejemplo: Podríamos brindarle a un LLM una herramienta para buscar en internet. Esta tool le permitirá realizar búsquedas en sitios webs para responder una pregunta hecha por el usuario o resolver una tarea que requiera información actualizada.



3. ¿Qué elementos componen un agente?

Aunque las arquitecturas pueden variar, un agente moderno basado en un LLM suele incluir lo siguiente:

- Modelo de IA: Interpreta la solicitud y genera decisiones o respuestas.
- Instrucciones: Definen el rol, el objetivo y las restricciones de un agente.
- Herramientas(tools): Permiten consultar datos o ejecutar operaciones.
- Contexto: Proporciona información relevante para una tarea actual.
- Estado o memoria: Permite conservar información entre pasos o interacciones, según la implementación.
- Controles: Establecen límites, validaciones y permisos.

No todos los agentes necesitan de estos componentes, por ejemplo, un agente sencillo podría funcionar sin memoria persistente y con una única herramienta.

Para entender un poco mejor esto, imaginemos que tenemos un asistente de ventas.

El usuario Xai consulta:

- “¿Cuál fue la categoría de productos que generó más ingresos durante septiembre y cuánto facturó?”

Un Chabot convencional podría explicar cómo calcular esta métrica o incluso intentar responder a partir de información que ya conoce, pero no tendría acceso automático a los datos reales de la empresa. Ahí es donde entra el agente.

Un agente correctamente configurado podría resolver el problema de la siguiente forma:

1. Interpreta la solicitud y la métrica consultada.
2. Determina que necesita consultar de la base de datos.
3. Utiliza una herramienta autorizada para obtener las ventas agrupadas por categoría
4. Compara los resultados obtenidos.
5. Responde la categoría de mayor facturación y su importe.

El agente no necesita conocer previamente los datos, y es que es la herramienta la que le proporcionará la información real que luego utilizará para generar la respuesta.

4. Implementación de un Agente utilizando ADK

Documentación: ADK DOCS
Google ADK (Agent Development Kit) es un framework de desarrollo de agentes que permite definir su comportamiento mediante código Python, TS, GO, Java y Kotlin.

ADK está pensado para ser utilizado con Gemini, pero también es posible utilizar otros proveedores con LiteLLM.

LiteLLM es una biblioteca que unifica la forma de comunicarse con numerosos proveedores de modelos. En lugar de programar una integración completamente distinta para cada proveedor, podemos utilizar una interfaz común.

Preparación del entorno:

1. Creamos y montamos el entorno virtual:

```
Shell
uv venv env && env\Scripts\activate #En Windows
```

o

```
Shell
uv venv env && env/bin/activate #En Linux
```

2. Instalamos google adk y litellm

```
Shell
uv pip install google-adk "litellm==1.105.1"
```

3. Comprobamos que ollama está activo:

```
Shell
ollama ls
```

Este último debería mostrarnos un listado de los modelos que tenemos actualmente en nuestra computadora.

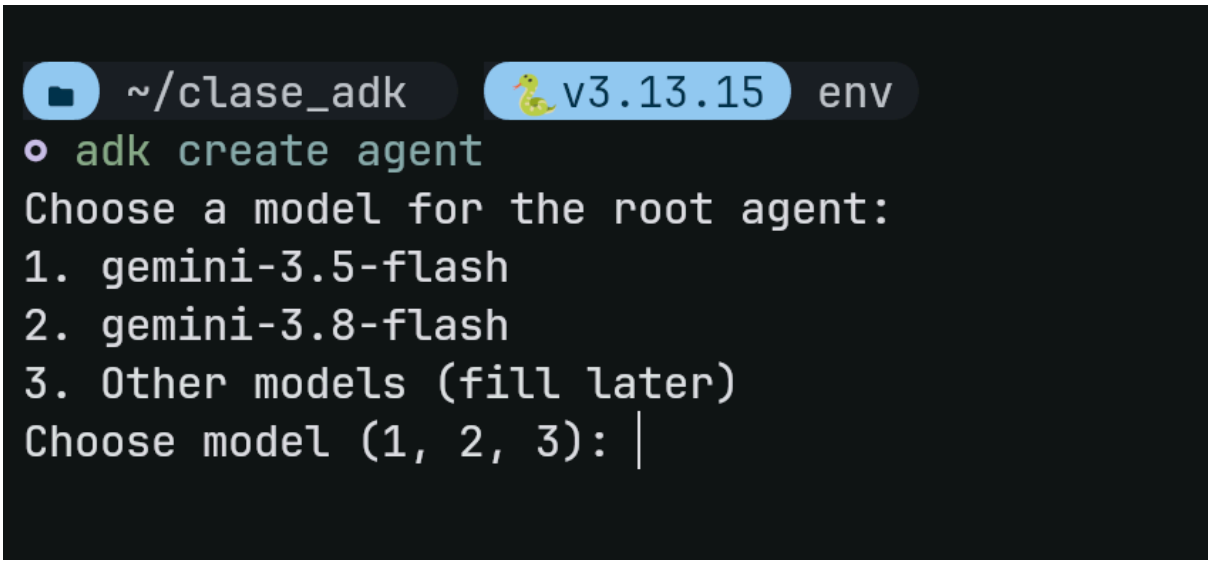
Creación de nuestro primer agente con ADK

Ahora construiremos nuestro primer agente funcional. El objetivo de nuestro agente será utilizar una herramienta que haga consultas a internet.

Ahora, vamos a ejecutar el siguiente comando:

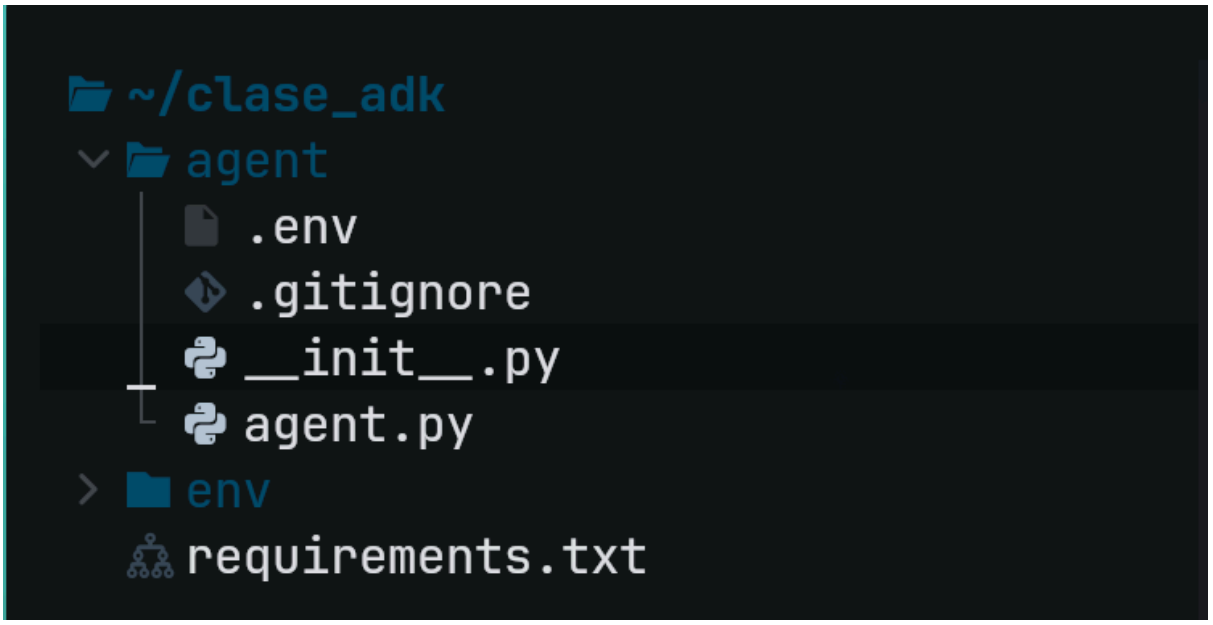
```
Shell
adk create agent
```

Al ejecutarlo, nos saldrá lo siguiente:



Como vamos a utilizar Ollama como nuestro proveedor, vamos a seleccionar la opción 3. (Si van a utilizar Gemini en vez de ollama, seleccionan la opción 1 o 2, no cambia mucho el tutorial).

Al finalizar, obtendremos lo siguiente:



Dentro del archivo agent.py, tendremos lo siguiente:

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='<FILL_IN_MODEL>',
    name='root_agent',
    description='A helpful assistant for user questions.',
    instruction='Answer user questions to the best of your knowledge',
)
```

Como se observa, al ejecutar el comando adk create "nombre_proyecto", se genera automáticamente una estructura base lista para empezar a trabajar.

Configuración del Modelo.

Como mencionamos anteriormente, utilizaremos LiteLLM para conectar nuestros modelos de Ollama en ADK. Antes de continuar, necesitamos declarar una variable de entorno para LiteLLM:

```python
import os
os.environ["OLLAMA_API_BASE"] = "http://localhost:11434"
```

Ahora, configuramos nuestro modelo:

```python
import os

os.environ["OLLAMA_API_BASE"] = "http://localhost:11434"

from google.adk.agents.llm_agent import Agent
from google.adk.models.lite_llm import LiteLlm

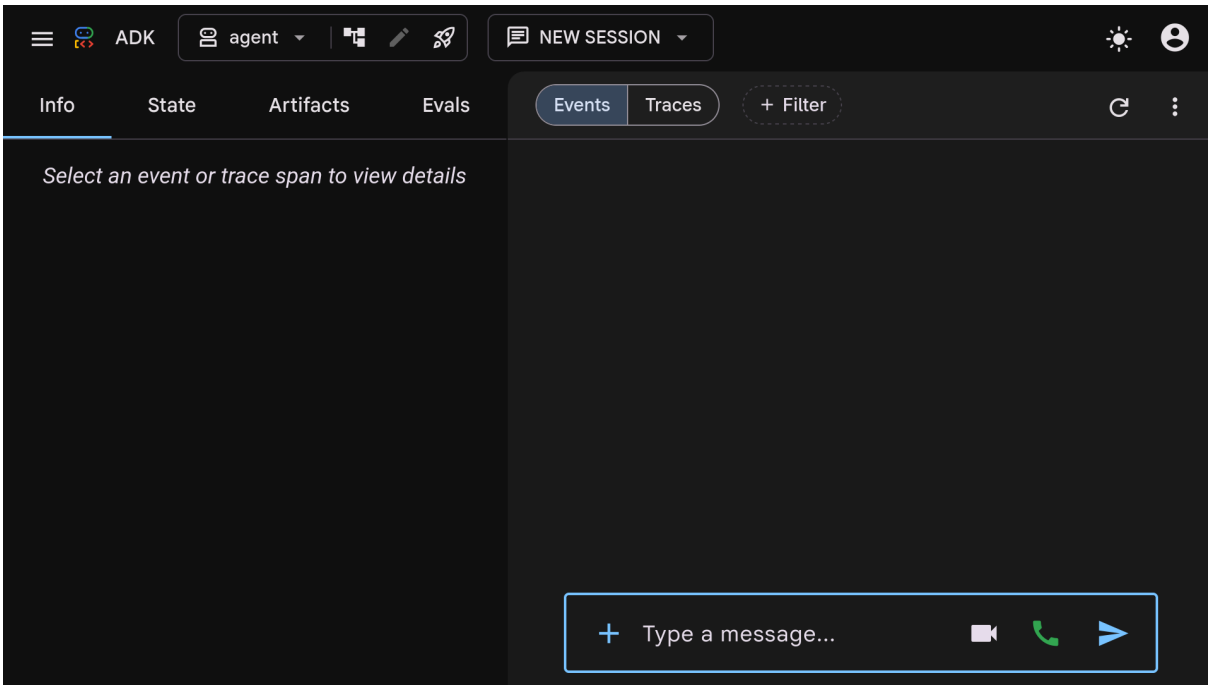
root_agent = Agent(
    model=LiteLlm("ollama_chat/gemma4:31b-cloud"),
```

```
    name="root_agent",
    description="Un asistente util que responde preguntas ",
    instruction="Eres un asistente util, que responde pregunta
sobre redes neuronales",
)
```

Con estas configuración ya podemos montar un servidor web y probar nuestro agente, para eso, vamos a ejecutar el siguiente comando:

```
Shell
adk web
```

Este comando ejecutará un servidor web en el puerto 8000, que si ingresamos desde el navegador, podremos visualizar una interfaz como esta:



Con esto ya podemos empezar a probar nuestro agente.

Vamos a crear nuestra primera herramienta. Utilizaremos la biblioteca de Duck Duck Go para crear una tool de web search:

1. Instalamos la biblioteca:

```shell
uv pip install ddgs
```

2. Creamos una función que reciba una query y un máximo de resultados:

```python
from ddgs import DDGS

def web_search(query: str, max_results: int = 5):
    """
    Herramienta para busquedas en internet utilizando
    duckduckgo_search
    Args:
        query: str -> La consulta buscar en internet.
        max_results: int -> cantidad maxima de resultados.
    """
    try:
        results = DDGS().text(query=query,
max_results=max_results)
        return results
    except Exception as e:
        print("Error interno al utilizar la herramienta", e)
        return str(e)
```

Nota: El docstring es fundamental al definir una función que actuará como herramienta (tool) para el modelo. Esta documentación especifica la finalidad de la función, sirviendo como la descripción que el modelo examinará para comprender cuándo utilizarla y qué argumentos recibe.

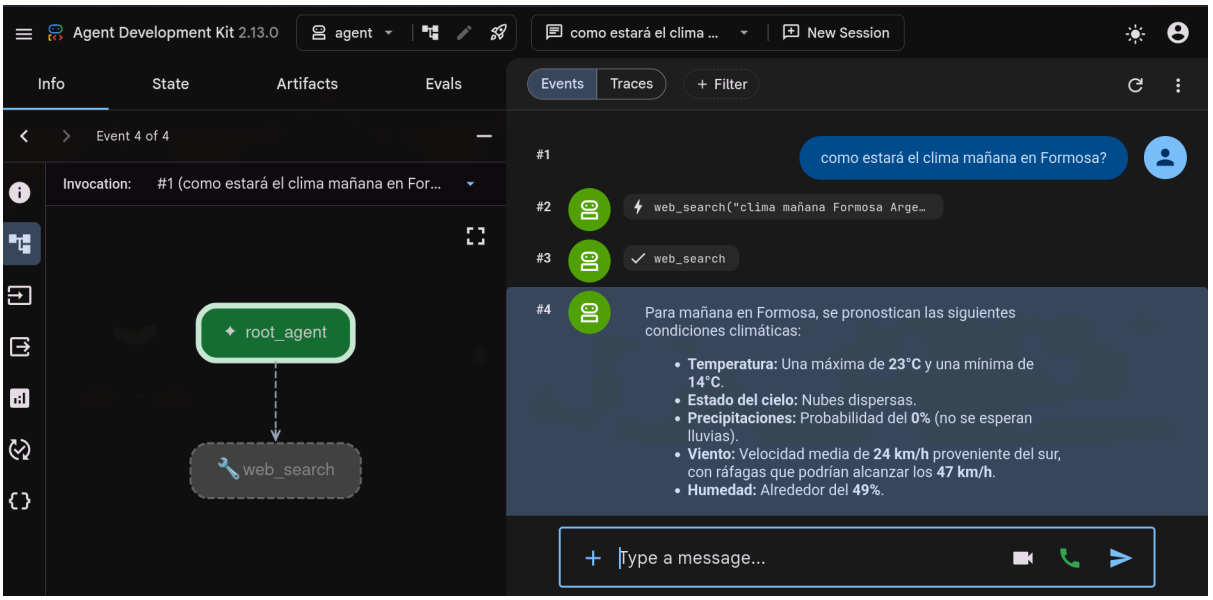
Y ahora agregamos un parámetro más de nuestro Agent y lo agregamos en la instrucción:

```python
root_agent = Agent(
    model=LiteLlm("ollama_chat/gemma4:31b-cloud"),
    name="root_agent",
    description="Un asistente util que responde preguntas ",
    instruction="Eres un asisistente util que responde preguntas sobre programacion. Utiliza tu herramienta llamada web_search para responder.",
    tools=[web_search],
)
```

Y ejecutamos nuevamente el comando:

```shell
adk web
```

Probamos nuestro agente:



Como podemos observar, ahora nuestro agente utiliza la herramienta que le dimos para responder preguntas.

5. Incorporación de múltiples herramientas

Hasta el momento, construimos un agente capaz de consultar información a internet mediante una herramienta de búsqueda. Sin embargo, los agentes no están limitados a utilizar una única herramienta, podemos proporcionarles distintas funciones para ampliar sus capacidades.

Vamos a crear una herramienta que indexe datos en una base de datos vectorial y una herramienta para recuperar información. Para ello, vamos a reutilizar algunos métodos y clases que vimos en la clase pasada con langchain.

Herramienta para indexar datos:

Python

```python
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_text_splitters import
RecursiveCharacterTextSplitter

def index_data(query: str, collection_name: str):
    """
    Herramienta para indexar data en la base de datos vectorial,
    Args:
        query: str -> Texto a indexar
        collection_name: str -> Nombre de la colección a utilizar
    """
    try:
        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=50,
        )
        docs = text_splitter.create_documents([query])
        vector_store = Chroma(
            embedding_function=OllamaEmbeddings(
                model="nomic-embed-text",
            ),
            persist_directory="./vector_store",
            collection_name=collection_name,
        )
        results = vector_store.add_documents(documents=docs)
```

```
        print("Documentos vectorizados: ", results)
        return f"Documentos vectorizados: {len(results)}"
    except Exception as e:
        return str(e)
```

Herramienta para listar colecciones:

```python
import chromadb

def list_collection():
    """
    Herramienta para listar las colleciones disponibles.
    """
    try:
        client = chromadb.PersistentClient(path="./vector_store")
        collections = client.list_collections()
        print(collections)
        return [c.name for c in collections]
    except Exception as e:
        return str(e)
```

Herramienta para la recuperación:

```python

def retrieval(query: str, collection_name: str, k: int = 5):
    """
    Herramienta para consultar la base de datos vectorial
    Args:
        query: str -> consulta
        collection_name: str -> nombre de la coleccion a
utilizar.
        k:int -> cantidad de documentos a obtener de la busqueda
    """
    try:
```

```
        vector_store = Chroma(
            embedding_function=OllamaEmbeddings(
                model="nomic-embed-text",
            ),
            persist_directory="./vector_store",
            collection_name=collection_name,
        )
        return
vector_store.similarity_search_with_relevance_scores(query=query,
k=k)
    except Exception as e:
        return str(e)
```

Ahora agregamos las herramientas que creamos a nuestro agente y cambiamos las instrucciones:

Python

```
import os
os.environ["OLLAMA_API_BASE"] = "http://localhost:11434"

from ddgs import DDGS
from google.adk.agents.llm_agent import Agent
from google.adk.models.lite_llm import LiteLlm
from tools.tools import index_data, list_collection, retrieval,
web_search

root_agent = Agent(
    model=LiteLlm("ollama_chat/gemma4:31b-cloud"),
    name="root_agent",
    description="Un asistente util que responde preguntas ",
    instruction="""Eres un asisistente util que responde
preguntas sobre programacion.
        Para responder una pregunta, utiliza primero la
herramienta list_collection para listar las colleciones
        luego utiliza la herramienta retrieval para obtener el
contexto necesario para responder.
```

```
        Si la información en la base de datos vectorial no es
suficiente, utiliza la herramienta web_search para buscar en
internet
        Cuanto se te pida guardar data en la base de datos,
        utiliza la herramienta llamada: index_data
        """,
    tools=[web_search, index_data, retrieval, list_collection],
)
```

Ahora, al hacer una pregunta, nuestro agente decidirá qué herramienta utilizar para poder respondernos:



Actividad Práctica:

Desarrollar un agente de inteligencia artificial utilizando Python y Google Agent Development Kit (ADK), capaz de resolver un problema concreto mediante el uso de herramientas.
El agente deberá interpretar las solicitudes del usuario, determinar qué acciones necesita realizar, utilizar las herramientas disponibles y construir una respuesta a partir de los resultados obtenidos.
No se considerará suficiente implementar un chatbot que únicamente genere respuestas mediante un modelo de lenguaje.

1. Modelo de IA: utilizar un modelo compatible con Google ADK, mediante LiteLLM u otra configuración trabajada en clase.

2. Instrucciones: definir el rol, el objetivo, las restricciones y el comportamiento esperado del agente.

3. Herramientas: implementar al menos tres herramientas con responsabilidades diferentes.

4. Uso de herramientas: demostrar que el agente puede seleccionar herramientas según la solicitud recibida.

5. Validación: contemplar errores, argumentos inválidos y resultados vacíos.

6. Interfaz de prueba: ejecutar el agente mediante la interfaz web de ADK o una alternativa acordada.

Importante: La entrega se debe realizar en un repositorio de github con la siguiente nomenclatura: agent_adk_nombre_apellido, con su correspondiente README.md explicando de que se trata, que herramientas tiene y cómo utilizarlo.